

1. ユーザー認証編

対象範囲

1. 会員新規登録
2. ログイン
3. Refresh Token再発行
4. ログアウト
5. Rate Limit
6. CSRF
7. 現在ユーザー取得
8. 暫定トップページ
9. React Router
10. AuthContext
11. ブラウザでの認証フロー確認

次の機能は別仕様書で扱う。

要件上の機能順

1. 会員新規登録
2. ログイン
3. ログアウト
4. Rate Limit
5. CSRF
6. 暫定トップページ
7. React Router
8. AuthContext
9. ブラウザ確認

実際の実装順

コードは依存関係に従い、次の順番で作成する。

Backend

1. Entityを作成する。
2. DB接続を作成する。
3. Migrationを作成する。
4. Repositoryを作成する。
5. Usecaseを作成する。
6. Controllerを作成する。
7. Middlewareを作成する。
8. Validatorを作成する。
9. Routerを作成する。
10. main.goで依存関係を接続する。
11. Backendのテストを実行する。
12. Backend APIを確認する。

Frontend

1. Frontendの型を作成する。
2. API Clientを作成する。
3. User APIを作成する。
4. AuthContextを作成する。
5. ProtectedRouteを作成する。
6. 会員登録画面を作成する。
7. ログイン画面を作成する。
8. 暫定トップページを作成する。
9. Not Found画面を作成する。
10. React Routerを作成する。
11. App.tsxへAuthContextとRouterを接続する。
12. Frontendのテストを実行する。
13. BackendとFrontendを起動する。
14. ブラウザで認証フローを確認する。

仕様書の層説明は、次の順番で記載する。

1. Entity
2. DB
3. Repository
4. Usecase

5. Controller
 6. Middleware
 7. Validator
 8. Router
-

完成状態

ユーザー編の実装が完了すると、ブラウザから次を確認できる。

1. 正しい入力で会員登録できる。
 2. 登録済みユーザーでログインできる。
 3. ログイン後に暫定トップページへ遷移する。
 4. 暫定トップページにユーザー情報が表示される。
 5. Access Tokenを使用して認証必須APIへアクセスできる。
 6. Refresh TokenからAccess Tokenを再発行できる。
 7. Refresh Tokenが再発行ごとに交換される。
 8. 使用済みRefresh Tokenを再利用できない。
 9. ページ再読み込み後に認証状態を復元できる。
 10. 未認証ユーザーは暫定トップページを表示できない。
 11. ログアウト後に同じRefresh Tokenを利用できない。
 12. 不正なCSRF TokenによるRefreshとLogoutを拒否できる。
 13. Rate Limit超過時にUserUsecaseへ到達する前に拒否できる。
 14. Stack Trace、Password、Token、CookieをAPIレスポンスへ含めない。
-

全体フロー

1. ゲストが会員登録画面を開く。
2. 名前、メールアドレス、パスワードを入力する。
3. Frontendが会員登録APIへ送信する。
4. Backendが入力内容を検証する。
5. Passwordをハッシュ化する。
6. `role = user`、`status = active`としてUserを保存する。
7. Frontendがログイン画面へ遷移する。
8. ユーザーがメールアドレスとパスワードを入力する。
9. FrontendがログインAPIへ送信する。

10. Backendが認証情報とUser状態を確認する。
 11. Access Token、Refresh Token、CSRF Tokenを発行する。
 12. Access TokenをJSONで返す。
 13. Refresh TokenをHttpOnly Cookieへ保存する。
 14. CSRF TokenをCookieへ保存する。
 15. AuthContextがAccess TokenとUserを保持する。
 16. React Routerが暫定トップページを表示する。
 17. Access Tokenの期限切れ時にRefresh APIを呼び出す。
 18. Refresh Tokenをローテーションする。
 19. 新しいAccess TokenをAuthContextへ保存する。
 20. ログアウト時にRefresh Token系列を失効する。
 21. Refresh Token CookieとCSRF Cookieを削除する。
 22. AuthContextを未認証状態へ戻す。
 23. ログイン画面へ遷移する。
-

Clean Architectureの処理順

HTTPリクエストは、原則として次の順序で処理する。

1. ReactがAPIへリクエストする。
2. RouterがURLとHTTPメソッドを判定する。
3. 共通Middlewareを実行する。
4. Route別Middlewareを実行する。
5. ControllerがJSON、Header、Cookieを受け取る。
6. ControllerがValidatorへ入力検証と正規化を依頼する。
7. Controllerが検証済みの値をUsecaseへ渡す。
8. Usecaseが業務ルールを判定する。
9. UsecaseがRepositoryへ保存または取得を依頼する。
10. RepositoryがPostgreSQLまたはRedisを操作する。
11. Repositoryが結果をUsecaseへ返す。
12. Usecaseが業務結果をControllerへ返す。
13. ControllerがHTTPレスポンスへ変換する。
14. Reactがレスポンスを受け取り、画面状態を更新する。

各層の責務

層	責務
Entity	UserとRefreshTokenの状態、型、業務ルールを保持する
DB	PostgreSQL接続とMigrationを管理する
Repository	DB・Redisへの保存、取得、排他制御、Transactionを扱う
Usecase	会員登録、認証、Token処理を組み立てる
Controller	HTTP入力、Cookie、HTTPレスポンスを扱う
Middleware	認証、CSRF、Rate Limitなどの横断処理を扱う
Validator	入力値の形式検証と正規化を行う
Router	URL、HTTPメソッド、Middleware、Controllerを接続する
Frontend API	Backend APIへの通信をまとめる
AuthContext	認証状態をFrontend全体で共有する
React Router	URL、画面、認証条件を接続する

7. ファイル構成

Backend

```

backend/
├── entity/
│   ├── user.go
│   ├── refresh_token.go
│   └── errors.go
├── db/
│   └── db.go
├── repository/
│   ├── user_repository.go
│   ├── refresh_token_repository.go
│   └── ratelimit_repository.go
├── usecase/
│   ├── user_usecase.go
│   └── token.go
├── controller/
│   └── user_controller.go
├── middleware/
│   ├── auth_middleware.go
│   ├── csrf_middleware.go
│   └── ratelimit_middleware.go
├── validator/
│   └── user_validator.go
├── router/
│   └── router.go
├── migrate/
│   ├── users.go
│   └── refresh_tokens.go
└── main.go

```

Frontend

```

frontend/src/
├── api/

```

```
|   | client.ts
|   | user.ts
|   | auth/
|   |   | AuthContext.tsx
|   |   | ProtectedRoute.tsx
|   | pages/
|   |   | SignupPage.tsx
|   |   | LoginPage.tsx
|   |   | TemporaryHomePage.tsx
|   |   | NotFoundPage.tsx
|   | router/
|   |   | AppRouter.tsx
|   | types/
|   |   | user.ts
|   | App.tsx
```

Entity仕様

User

役割

Userは次の情報を保持する。

- 名前
- メールアドレス
- ハッシュ済みPassword
- UserのRole
- UserのStatus
- Access Token一括無効化用のTokenVersion
- 作成日時
- 更新日時

UserRole

値	内容
user	一般ユーザー
admin	管理者

一般会員登録では、必ず `user` を設定する。会員登録RequestからRoleを受け取らない。

UserStatus

値	内容
active	ログインと認証必須機能を利用できる
suspended	ログイン、Token再発行、認証必須機能を利用できない

フィールド

フィールド	型	仕様
ID	uint64	DBが発行する識別子
Name	string	trim後1~50文字
Email	string	trim・小文字化して保存する
PasswordHash	string	ハッシュ済みPasswordだけを保存する
Role	UserRole	一般登録時は user
Status	UserStatus	登録時は active
TokenVersion	uint64	Access Token一括無効化用。初期値0
CreatedAt	time.Time	作成日時
UpdatedAt	time.Time	最終更新日時

Userで行う判定

操作	内容
IsActive	Statusが active か判定する
IsAdmin	Roleが admin か判定する
InvalidateAccessTokens	TokenVersionを1増加する

RefreshToken

役割

RefreshTokenは次の状態を保持する。

- Tokenの所有User
- TokenのHash
- Token系列
- 使用済み状態
- 失効状態
- 有効期限
- 次に発行したToken

平文Refresh TokenはEntity、DB、ログへ保存しない。

フィールド

フィールド	型	仕様
ID	uint64	DBが発行する識別子
UserID	uint64	Tokenを所有するUser ID

フィールド	型	仕様
TokenHash	string	平文Tokenから生成したHash
FamilyID	string	同じログイン系列を識別する値
ReplacedByID	*uint64	次に発行したRefreshToken ID
ExpiresAt	time.Time	有効期限
UsedAt	*time.Time	Token再発行へ使用した日時
RevokedAt	*time.Time	ログアウト等で失効した日時
CreatedAt	time.Time	発行日時

RefreshTokenで行う判定

操作	内容
IsExpired	有効期限を過ぎているか判定する
IsUsable	未使用、未失効、期限内か判定する
MarkUsed	UsedAtとReplacedByIDを設定する
Revoke	RevokedAtを設定する

同じFamilyIDの一括失効は、Usecaseが必要性を判断し、RepositoryがDB更新を行う。

DB仕様

共通ルール

項目	仕様
DB	PostgreSQL
日時	TIMESTAMPZ
アプリ内部時刻	UTC
テーブル名	複数形のsnake_case
カラム名	snake_case
主キー	正数のBIGINT
PostgreSQL ENUM	使用しない
Enum	文字列とCHECK制約
Migration	MigrationはGoで実装する。 Migrationは専用コマンドまたは明示的な実行処理から起動する。 main.goの通常起動時には無条件実行しない。
AutoMigrate	本番の正規Migrationとして使用しない
updated_at	DBトリガーで自動更新しない

users

カラム	NULL	制約
<code>id</code>	不可	主キー
<code>name</code>	不可	1~50文字
<code>email</code>	不可	254文字以内、重複不可
<code>password_hash</code>	不可	平文保存禁止
<code>role</code>	不可	<code>user</code> または <code>admin</code>
<code>status</code>	不可	<code>active</code> または <code>suspended</code>
<code>token_version</code>	不可	0以上、初期値0
<code>created_at</code>	不可	作成日時
<code>updated_at</code>	不可	最終更新日時

制約

- `email` へUNIQUE制約を設定する。
- `role` へ `user` または `admin` のCHECK制約を設定する。
- `status` へ `active` または `suspended` のCHECK制約を設定する。
- `token_version` へ0以上のCHECK制約を設定する。

Email重複はUsecaseで事前確認し、同時実行時の重複はDBのUNIQUE制約で防止する。

refresh_tokens

カラム	NULL	制約
<code>id</code>	不可	主キー
<code>user_id</code>	不可	<code>users.id</code> を参照
<code>token_hash</code>	不可	重複不可
<code>family_id</code>	不可	Token系列
<code>replaced_by_id</code>	可	<code>refresh_tokens.id</code> を自己参照
<code>expires_at</code>	不可	有効期限
<code>used_at</code>	可	未使用時はNULL
<code>revoked_at</code>	可	有効時はNULL
<code>created_at</code>	不可	発行日時

必要なIndex

対象	目的
<code>users.email</code>	会員登録とログイン
<code>refresh_tokens.token_hash</code>	Token検索

対象	目的
<code>refresh_tokens.user_id</code>	User単位の失効
<code>refresh_tokens.family_id</code>	Token系列の一括失効
<code>refresh_tokens.expires_at</code>	期限切れTokenの削除

Repository仕様

共通方針

- GORMはRepository内だけで扱う。
- Usecase、Controller、Middlewareへ `gorm.DB` を渡さない。
- SQL文字列へUser入力を直接連結しない。
- GORMのプレースホルダーを使用する。
- `TxManager` は作成しない。
- Transactionが必要な処理は、用途が明確なRepositoryメソッド内で行う。
- UsecaseはBegin、Commit、Rollbackを直接扱わない。

UserRepository

メソッド	役割
<code>Create</code>	Userを作成する
<code>FindByEmail</code>	EmailからUserを取得する
<code>FindByID</code>	User IDからUserを取得する
<code>Update</code>	User状態やTokenVersionを更新する

責務

Create

- UserをDBへ保存する。
- EmailのUNIQUE違反を判定する。
- PasswordHashだけを保存する。
- 平文Passwordを受け取らない。

FindByEmail

- 正規化済みEmailからUserを取得する。
- LoginとEmail重複確認に使用する。

FindByID

- Access Token内のUser IDからUserを取得する。
- 現在のStatusとTokenVersionの確認に使用する。

Update

- UserのStatusやTokenVersionを更新する。
- `updated_at` をアプリ側で更新する。

RefreshTokenRepository

メソッド	役割
<code>Create</code>	Login時にRefreshTokenを作成する
<code>FindByTokenHash</code>	HashからRefreshTokenを取得する
<code>Rotate</code>	旧Tokenの排他取得、新Token作成、旧Token使用済み化を同じTransactionで行う
<code>RevokeFamily</code>	同じFamilyIDのTokenを一括失効する
<code>RevokeFamilyAndIncrementTokenVersion</code>	再利用検知時にFamily失効とUser.TokenVersion更新を同じTransactionで行う
<code>RevokeByUserID</code>	Userに属するRefresh Tokenを一括失効する
<code>DeleteExpired</code>	期限切れTokenを削除する

Create

Login成功時に、RefreshTokenを1件作成する。

FindByTokenHash

Cookieから受け取った平文Refresh TokenをHash化した値で、対象Tokenを取得する。

Rotate

次を同じDB Transactionで処理する。

1. 旧Refresh TokenをTokenHashから排他取得する。
2. 旧Refresh Tokenの状態を再確認する。
3. 新しいRefresh Tokenを作成する。
4. 旧Refresh TokenへUsedAtを設定する。
5. 旧Refresh TokenへReplacedByIDを設定する。
6. すべて成功した場合はCommitする。
7. 途中で失敗した場合はRollbackする。

同じRefresh Tokenを複数のRequestが同時に利用できないよう、旧Refresh Tokenを行ロックする。

RevokeFamily

Logout時に、同じFamilyIDの未失効Refresh TokenへRevokedAtを設定する。

RevokeFamilyAndIncrementTokenVersion

Refresh Token再利用検知時に、次を同じDB Transactionで処理する。

1. 対象Userを排他取得する。
2. 同じFamilyIDのRefresh Tokenをすべて失効する。
3. User.TokenVersionを1増加する。
4. User.UpdatedAtを更新する。
5. すべて成功した場合はCommitする。
6. 途中で失敗した場合はRollbackする。

RevokeByUserID

User利用停止時に、そのUserに属するRefresh Tokenを失効する。

DeleteExpired

期限切れRefresh Tokenを削除する。実行方法と実行時刻は、このユーザー編では固定しない。

RateLimitRepository

メソッド	役割
<code>Allow</code>	RedisのLua Scriptを実行してToken Bucketを判定する

Repositoryは次を担当する。

- Redis Keyの読み書き
- 現在Token数の取得
- 経過時間によるToken補充
- Request分のToken消費
- 更新時刻の保存
- TTLの設定
- 判定結果の返却

Tokenの取得、補充、消費、保存はLua Script内で原子的に行う。

Usecase仕様

UserUsecase

ユーザー認証を `usecase/user_usecase.go` へまとめる。

メソッド	内容
<code>SignUp</code>	会員登録
<code>Login</code>	Password照合とToken発行
<code>Refresh</code>	Refresh Tokenローテーション
<code>Logout</code>	Refresh Token失効
<code>GetMe</code>	現在User取得
<code>ValidateTokenVersion</code>	JWT内とDB内のTokenVersionを比較する

UserUsecaseは次を扱わない。

- HTTP Status
- Echo Context
- Cookieの設定・削除
- Request Header
- Response Header
- GORM
- Redisコマンド
- TransactionのBegin・Commit・Rollback

SignUp

1. Controllerから検証済みのName、Email、Passwordを受け取る。
 2. EmailからUserを検索する。
 3. 同じEmailのUserが存在する場合は重複エラーを返す。
 4. Passwordを安全な方式でHash化する。
 5. Roleを `user` に設定する。
 6. Statusを `active` に設定する。
 7. TokenVersionを0に設定する。
 8. UserRepository.Createを呼び出す。
 9. 作成したUserをControllerへ返す。
- 一般会員登録からadminを作成しない。

Login

1. Controllerから検証済みのEmailとPasswordを受け取る。
 2. UserRepository.FindByEmailを呼び出す。
 3. Userが存在しない場合は認証失敗として扱う。
 4. 入力PasswordとPasswordHashを照合する。
 5. User.Statusが `active` であることを確認する。
 6. Access Tokenを生成する。
 7. 平文Refresh Tokenを生成する。
 8. Refresh TokenのHashを生成する。
 9. FamilyIDを生成する。
 10. RefreshToken Entityを作成する。
 11. RefreshTokenRepository.Createを呼び出す。
 12. CSRF Tokenを生成する。
 13. Access Token、平文Refresh Token、CSRF Token、UserをControllerへ返す。
- Emailが存在しない場合とPasswordが異なる場合で、利用者向けエラーを分けない。
-

Refresh

1. Controllerから平文Refresh Tokenを受け取る。
2. 平文Refresh TokenをHash化する。
3. RefreshTokenRepository.FindByTokenHashを呼び出す。
4. Tokenが存在することを確認する。
5. Tokenが期限切れでないことを確認する。
6. Tokenが失効済みでないことを確認する。
7. Tokenが使用済みか確認する。
8. Tokenに対応するUserを取得する。
9. User.Statusが `active` であることを確認する。

未使用Tokenの場合

1. 新しい平文Refresh Tokenを生成する。
2. 新しいRefresh Token Hashを生成する。
3. 同じFamilyIDを引き継いだRefreshTokenを作成する。
4. RefreshTokenRepository.Rotateを呼び出す。
5. Rotateが排他制御とTransactionを実行する。

6. 新しいAccess Tokenを生成する。
7. 新しいCSRF Tokenを生成する。
8. 新しいAccess Token、Refresh Token、CSRF TokenをControllerへ返す。

使用済みTokenの場合

1. Token再利用として判定する。
2. RefreshTokenRepository.RevokeFamilyAndIncrementTokenVersionを呼び出す。
3. 同じFamilyIDのRefresh Tokenを失効する。
4. User.TokenVersionを1増加する。
5. Access Token再発行を拒否する。

Usecaseが事前確認した後も、Rotateでは行ロック取得後のToken状態を再確認する。

Logout

1. Controllerから平文Refresh Tokenを受け取る。
2. 平文Refresh TokenをHash化する。
3. RefreshTokenRepository.FindByTokenHashを呼び出す。
4. Tokenが存在する場合はFamilyIDを取得する。
5. RefreshTokenRepository.RevokeFamilyを呼び出す。
6. 同じFamilyIDのRefresh Tokenを失効する。
7. Controllerへ処理結果を返す。

既に失効済みのTokenに対するLogoutは、再送可能な処理として成功扱いにする。LogoutではUser.TokenVersionを変更しない。

GetMe

1. Auth MiddlewareからUser IDを受け取る。
 2. UserRepository.FindByIDを呼び出す。
 3. Userが存在することを確認する。
 4. User.Statusが **active** であることを確認する。
 5. UserをAuth Middlewareへ返す。
-

ValidateTokenVersion

1. JWT内のTokenVersionを受け取る。
2. DBから取得したUser.TokenVersionと比較する。

3. 値が一致する場合は認証を継続する。
4. 値が異なる場合はAccess Tokenを無効として拒否する。

Token処理

`usecase/token.go` では、次の認証処理を扱う。

Password

- PasswordをHash化する。
- 入力PasswordとPasswordHashを照合する。
- 平文Passwordを保存しない。
- PasswordやPasswordHashをログへ出力しない。

Access Token

Access Tokenへ次を含める。

Claim	内容
<code>sub</code>	User ID
<code>role</code>	UserRole
<code>tv</code>	TokenVersion
<code>iat</code>	発行日時
<code>exp</code>	有効期限

次を行う。

- Access Token生成
- JWT署名検証
- 有効期限検証
- User ID取得
- Role取得
- TokenVersion取得

Refresh Token

次を行う。

- 安全な乱数から平文Refresh Tokenを生成する。
- DB保存用TokenHashを生成する。
- FamilyIDを生成する。
- DBにはTokenHashだけを保存する。
- 平文Refresh Tokenをログへ出力しない。

CSRF Token

次を行う。

- CSRF Tokenを生成する。
- CookieとHeaderの値を比較する。
- CSRF Tokenをログへ出力しない。

Rate Limit Usecase

Rate Limit Usecaseは、次を担当する。

処理	内容
API種別判定	会員登録、ログイン、Token再発行を判定する
制限設定選択	APIごとの制限値を選択する
Redis Key生成	API種別とIP識別値から作成する
Repository呼び出し	RateLimitRepository.Allowを呼び出す
結果返却	許可、残数、再試行までの時間を返す

Controller仕様

UserController

ユーザー認証用Controllerを `controller/user_controller.go` へまとめる。

メソッド	内容
<code>SignUp</code>	会員登録Requestを受け取る
<code>Login</code>	Login処理を呼び、CookieとJSONを返す
<code>Refresh</code>	Refresh Token Cookieを取得して再発行する
<code>Logout</code>	Logout処理を呼び、Cookieを削除する
<code>Me</code>	現在User情報を返す

Controllerは次を行わない。

- Password照合
- Password Hash化
- JWT生成
- Refresh Token Hash化
- DB操作
- Token再利用検知
- Transaction制御

SignUp

1. Request Bodyを受け取る。
 2. JSONをRequest用の型へBindする。
 3. UserValidator.ValidateSignupを呼び出す。
 4. 検証済みの値をUserUsecase.SignUpへ渡す。
 5. UserをResponseへ変換する。
 6. **201 Created** を返す。
-

Login

1. Request Bodyを受け取る。
 2. JSONをRequest用の型へBindする。
 3. UserValidator.ValidateLoginを呼び出す。
 4. 検証済みの値をUserUsecase.Loginへ渡す。
 5. Refresh TokenをHttpOnly Cookieへ設定する。
 6. CSRF TokenをCookieへ設定する。
 7. Access TokenとUser情報をJSONで返す。
 8. **200 OK** を返す。
-

Refresh

1. Refresh Token Cookieを取得する。
 2. Cookieが存在しない場合は認証エラーを返す。
 3. UserUsecase.Refreshを呼び出す。
 4. 正常時は新しいRefresh Token Cookieを設定する。
 5. 正常時は新しいCSRF Cookieを設定する。
 6. 新しいAccess TokenをJSONで返す。
 7. Token再利用検知時は認証Cookieを削除する。
 8. **200 OK** または認証エラーを返す。
-

Logout

1. Refresh Token Cookieを取得する。
2. UserUsecase.Logoutを呼び出す。
3. Refresh Token Cookieを削除する。

4. CSRF Cookieを削除する。
 5. `204 No Content` を返す。
-

Me

1. Auth MiddlewareがContextへ保存したUserを取得する。
 2. UserをResponseへ変換する。
 3. `200 OK` を返す。
-

Middleware仕様

共通Middleware

共通Middlewareは次の順序で実行する。

1. Request ID
2. Recover
3. Logger
4. Security Header
5. CORS
6. Body Limit
7. Route別Middleware
8. Controller

Request ID

- Requestごとに一意のRequest IDを設定する。
- Responseとログへ同じRequest IDを記録する。
- APIエラーへRequest IDを含める。

Recover

- panicを捕捉する。
- Stack Traceをサーバーログへ記録する。
- Stack TraceをAPIレスポンスへ返さない。
- 500エラーへ変換する。

Logger

記録する。

- 時刻
- ログレベル
- Request ID
- User ID
- 処理名
- HTTP Status
- 処理結果

記録しない。

- Password
- PasswordHash
- Access Token
- Refresh Token
- CSRF Token
- Cookie
- 秘密鍵

Security Header

- CSP
- X-Content-Type-Options
- frame-ancestorsに相当する画面埋め込み対策

CORS

- 本番では許可したFrontend Originだけを許可する。
- Cookie送信に必要なCredentialsを許可する。
- 未許可OriginからのブラウザRequestを拒否する。

Body Limit

- JSON Requestへ容量上限を設定する。
- 上限を超えたRequestをControllerへ到達させない。
- 上限超過時は `413 Payload Too Large` を返す。

Rate Limit Middleware

対象

API	識別単位
会員登録	IPアドレス
ログイン	IPアドレス
Token再発行	IPアドレス

フロー

1. 対象APIへRequestが届く。
2. MiddlewareがIPアドレスを取得する。
3. Rate Limit UsecaseへAPI種別とIPアドレスを渡す。
4. Rate Limit Usecaseが制限設定を選択する。
5. RateLimitRepositoryがLua Scriptを実行する。
6. 制限内であればControllerへ進む。
7. 制限超過時はUserUsecaseへ到達させない。
8. `429 Too Many Requests` を返す。
9. `Retry-After` を設定する。

Redis Key

Redis Keyには次を含める。

- Rate Limit用Keyであること
- API種別
- IPを安全に変換した識別値

EmailやPasswordをRedis Keyへ含めない。

完了条件

- 制限内のRequestは処理される。
- 制限超過時は429になる。
- 制限超過時はUserUsecaseへ到達しない。
- 時間経過後にTokenが補充される。
- 同時Requestでも制限を超えて許可されない。
- Redis更新はLua Scriptで原子的に行われる。

CSRF Middleware

対象

API	CSRF
<code>/signup</code>	不要
<code>/login</code>	不要
<code>/refresh</code>	必要
<code>/logout</code>	必要
<code>/me</code>	不要

フロー

1. LoginまたはRefresh成功時にCSRF TokenをCookieへ保存する。
2. FrontendがCSRF Cookieを読み取る。
3. `X-CSRF-Token` Headerへ値を設定する。
4. CSRF MiddlewareがCookieとHeaderを取得する。
5. 両方が存在することを確認する。
6. 値が一致することを確認する。
7. 一致した場合だけControllerへ進む。
8. 未指定または不一致の場合は403を返す。
9. CSRF Tokenをログへ出力しない。

完了条件

- 正しいCSRF TokenでRefreshできる。
- 正しいCSRF TokenでLogoutできる。
- Cookieがない場合は403になる。
- Headerがない場合は403になる。
- 値が不一致の場合は403になる。
- 不正RequestはUserUsecaseへ到達しない。

Auth Middleware

フロー

1. Authorization Headerを取得する。
2. Bearer形式を確認する。
3. JWT署名を検証する。
4. JWT有効期限を確認する。
5. JWTからUser IDを取得する。

6. UserUsecase.GetMeを呼び出す。
 7. User.Statusが `active` であることを確認する。
 8. UserUsecase.ValidateTokenVersionを呼び出す。
 9. JWT内とDB内のTokenVersionを比較する。
 10. 正常なUserをEcho Contextへ保存する。
 11. Controllerへ進む。
- JWT内のRoleだけで現在のUser状態を確定しない。
-

Validator仕様

UserValidator

Validatorは `validator/user_validator.go` へまとめる。

メソッド	検証内容
<code>ValidateSignup</code>	Name、Email、Password
<code>ValidateLogin</code>	Email、Password

Validatorは次を行わない。

- DB検索
 - Email重複確認
 - Password Hash化
 - Token生成
 - HTTP Response作成
-

会員登録

項目	条件
Name	trim後1〜50文字
Email	trim・小文字化後254文字以内
Email形式	有効なメールアドレス形式
Password	8〜72文字
Role	Requestで受け取らない
Status	Requestで受け取らない

NameとEmailは正規化後の値をControllerへ返す。

Passwordはtrimしない。

Login

項目	条件
Email	必須、有効なメールアドレス形式
Password	必須
Email	trim・小文字化する
Password	trimしない

Router仕様

Routerには業務処理を書かない。

API一覧

HTTP	URL	内容	認証	Rate Limit	CSRF
POST	/signup	会員登録	不要	必要	不要
POST	/login	ログイン	不要	必要	不要
POST	/refresh	Access Token 再発行	Refresh Cookie	必要	必要
POST	/logout	ログアウト	Refresh Cookie	不要	必要
GET	/me	現在User取得	Access Token	不要	不要

Router接続

URL	Middleware	Controller
POST /signup	Rate Limit	SignUp
POST /login	Rate Limit	Login
POST /refresh	Rate Limit、CSRF	Refresh
POST /logout	CSRF	Logout
GET /me	Auth	Me

main.go仕様

main.goには業務処理を書かない。

依存関係の生成順

1. 環境変数を読み取る。
2. PostgreSQLへ接続する。
3. Redisへ接続する。
4. UserValidatorを生成する。

5. UserRepositoryを生成する。
 6. RefreshTokenRepositoryを生成する。
 7. RateLimitRepositoryを生成する。
 8. Token処理を生成する。
 9. UserUsecaseを生成する。
 10. Rate Limit Usecaseを生成する。
 11. UserControllerを生成する。
 12. Middlewareを生成する。
 13. Routerを生成する。
 14. HTTPサーバーを起動する。
 15. 終了シグナルを受け取る。
 16. Graceful Shutdownを実行する。
- 本番起動時にMigrationやSeedを無条件実行しない。

APIエラー仕様

すべてのAPIは同じ形式でエラーを返す。

項目	内容
<code>status</code>	HTTPステータス
<code>code</code>	Frontendが判定するエラーコード
<code>message</code>	利用者向けメッセージ
<code>request_id</code>	サーバーログとの紐付け

次をAPIレスポンスへ含めない。

- SQL
- Stack Trace
- Password
- PasswordHash
- Access Token
- Refresh Token
- CSRF Token
- Cookie
- 秘密鍵

会員新規登録仕様

フロー

1. ユーザーが会員登録画面を開く。
2. Name、Email、Passwordを入力する。
3. Frontendが `POST /signup` へ送信する。
4. Body Limit Middlewareが容量を確認する。
5. Rate Limit Middlewareが回数制限を確認する。
6. ControllerがRequest Bodyを受け取る。
7. Validatorが入力内容を検証する。
8. Nameの前後空白を除去する。
9. Emailの前後空白を除去し、小文字へ統一する。
10. Controllerが検証済みの値をUsecaseへ渡す。
11. Usecaseが同じEmailの登録有無を確認する。
12. PasswordをHash化する。
13. `role = user`、`status = active` のUserを作成する。
14. RepositoryがUserをDBへ保存する。
15. Controllerが作成結果を返す。
16. FrontendがLogin画面へ遷移する。

各層の実装

層	実装内容
Entity	Userの初期状態を保持する
DB	Email重複を禁止する
Repository	Create、FindByEmail
Usecase	重複確認、Password Hash化、User作成
Controller	Bind、Validator呼び出し、Usecase呼び出し、Response
Middleware	Body Limit、Rate Limit
Validator	Name、Email、Passwordを検証する
Router	<code>POST /signup</code> を登録する
Frontend	SignupPageとAPI通信を作成する

成功レスポンス

項目	内容
HTTP	<code>201 Created</code>
ID	作成したUser ID
Name	User名

項目	内容
Email	正規化済みEmail
Role	user
Status	active
CreatedAt	作成日時

PasswordとPasswordHashは返さない。

エラー

HTTP	条件
400	JSONまたは入力値が不正
409	Emailが登録済み
413	Request Bodyが大きすぎる
429	Rate Limit超過
500	予期しない内部エラー

完了条件

- 正しい入力でUserが1件作成される。
- Roleは常に user になる。
- Statusは常に active になる。
- Emailを重複登録できない。
- 平文PasswordがDBとログに残らない。
- 一般登録APIからadminを作成できない。
- ブラウザから登録できる。

Login仕様

フロー

1. ユーザーがLogin画面を開く。
2. EmailとPasswordを入力する。
3. Frontendが POST /login へ送信する。
4. Body Limit Middlewareが容量を確認する。
5. Rate Limit Middlewareが制限を確認する。
6. ControllerがRequest Bodyを受け取る。
7. ValidatorがEmailとPasswordを検証する。
8. Emailを正規化する。

9. Controllerが検証済みの値をUsecaseへ渡す。
10. UsecaseがEmailからUserを取得する。
11. 入力PasswordとPasswordHashを照合する。
12. User.Statusが `active` であることを確認する。
13. Access Tokenを生成する。
14. Refresh Tokenを生成する。
15. Refresh Token HashをDBへ保存する。
16. CSRF Tokenを生成する。
17. ControllerがRefresh TokenをHttpOnly Cookieへ設定する。
18. ControllerがCSRF TokenをCookieへ設定する。
19. Access TokenとUser情報をJSONで返す。
20. AuthContextが認証状態を保持する。
21. 暫定トップページへ遷移する。

各層の実装

層	実装内容
Entity	Userのactive判定、RefreshToken状態
DB	User取得、RefreshToken保存
Repository	FindByEmail、RefreshToken Create
Usecase	Password照合、Token発行、Token保存
Controller	Cookie設定、JSON Response
Middleware	Body Limit、Rate Limit
Validator	EmailとPasswordを検証する
Router	<code>POST /login</code>
Frontend	LoginPage、AuthContext更新

成功レスポンス

項目	内容
HTTP	<code>200 OK</code>
Access Token	JSONで返す
Token Type	Bearer
有効期限	秒数または期限日時
User	ID、Name、Email、Role、Status

Refresh Token Cookie

属性	仕様
HttpOnly	有効
Secure	本番で有効
SameSite	FrontendとAPIの配置に合わせる
Path	RefreshとLogoutへ送信可能な範囲
有効期限	Refresh Token期限と一致

CSRF Cookie

属性	仕様
HttpOnly	無効
Secure	本番で有効
SameSite	Refresh Token Cookieと揃える
有効期限	Refresh Token期限と一致

エラー

HTTP	条件
400	入力不正
401	EmailまたはPassword不正
403	suspended User
429	Rate Limit超過
500	Token生成またはDB失敗

完了条件

- 正しい認証情報でLoginできる。
- 誤った認証情報ではTokenが発行されない。
- suspended UserはLoginできない。
- Refresh TokenがHttpOnly Cookieへ保存される。
- Access TokenがFrontendへ返る。
- 暫定トップページへ遷移できる。

Refresh Token再発行 仕様

フロー

1. Access Tokenの期限が切れる。
2. FrontendがCSRF Cookieを読み取る。
3. `X-CSRF-Token` Headerへ値を設定する。

4. `POST /refresh` を呼び出す。
5. Rate Limit Middlewareが制限を確認する。
6. CSRF MiddlewareがCookieとHeaderを照合する。
7. ControllerがRefresh Token Cookieを取得する。
8. UsecaseがRefresh TokenをHash化する。
9. RepositoryからRefreshTokenを取得する。
10. UsecaseがUsedAt、RevokedAt、ExpiresAtを確認する。
11. Usecaseが対象Userを取得する。
12. User.Statusが `active` であることを確認する。
13. 未使用Tokenの場合は新Refresh Tokenを生成する。
14. UsecaseがRefreshTokenRepository.Rotateを呼び出す。
15. Rotateが旧Refresh Tokenを行ロックする。
16. RotateがToken状態を再確認する。
17. Rotateが新Refresh Tokenを保存する。
18. Rotateが旧Refresh TokenへUsedAtとReplacedByIDを設定する。
19. すべて成功した場合はCommitする。
20. 途中で失敗した場合はRollbackする。
21. Usecaseが新しいAccess Tokenを生成する。
22. Usecaseが新しいCSRF Tokenを生成する。
23. Controllerが新しいRefresh Token Cookieを設定する。
24. Controllerが新しいCSRF Cookieを設定する。
25. Access TokenをJSONで返す。
26. AuthContextがAccess Tokenを更新する。

再利用検知

1. UsedAtが設定済みのRefresh Tokenを検知する。
2. UsecaseがToken再利用として判定する。
3. RevokeFamilyAndIncrementTokenVersionを呼び出す。
4. Repositoryが対象Userを行ロックする。
5. Repositoryが同じFamilyIDのTokenを失効する。
6. RepositoryがUser.TokenVersionを1増加する。
7. 同じTransactionでCommitする。
8. Access Token再発行を拒否する。

9. Controllerが認証Cookieを削除する。

10. AuthContextを未認証状態へ戻す。

各層の実装

層	実装内容
Entity	IsExpired、IsUsable、MarkUsed、Revoke
DB	Refresh Token状態とUser.TokenVersionを保存する
Repository	行ロック、Rotate、Family失効、TokenVersion更新、Transaction
Usecase	Token状態を判定し、Rotationまたは再利用検知を選択する
Controller	Cookie取得、Cookie更新、Response
Middleware	Rate Limit、CSRF
Validator	Request Bodyはないため入力Validatorは使用しない
Router	POST /refresh
Frontend	Access Token更新、元Requestを1回だけ再送する

完了条件

- 正しいRefresh TokenでAccess Tokenを再発行できる。
- 再発行ごとにRefresh Tokenが交換される。
- 古いRefresh Tokenを再利用できない。
- 再利用検知後は同じFamilyを利用できない。
- 同時RefreshでTokenが二重発行されない。
- ページ再読み込み後に認証状態を復元できる。

Logout仕様

フロー

1. ユーザーがLogoutボタンを押す。
2. FrontendがCSRF Cookieを読み取る。
3. X-CSRF-Token Headerへ値を設定する。
4. POST /logout を呼び出す。
5. CSRF MiddlewareがCookieとHeaderを照合する。
6. ControllerがRefresh Token Cookieを取得する。
7. UsecaseがTokenをHash化する。
8. RepositoryがTokenを取得する。
9. Repositoryが同じFamilyIDのRefresh Tokenを失効する。

10. ControllerがRefresh Token Cookieを削除する。
11. ControllerがCSRF Cookieを削除する。
12. AuthContextがUserとAccess Tokenを削除する。
13. Login画面へ遷移する。
14. 同じRefresh TokenからAccess Tokenを再発行できないことを確認する。

各層の実装

層	実装内容
Entity	RefreshTokenのRevoke
DB	RevokedAtを保存する
Repository	Token取得、Family失効
Usecase	Logout処理
Controller	Cookie削除
Middleware	CSRF
Validator	Request Bodyはないため入力Validatorは使用しない
Router	POST /logout
Frontend	AuthContext初期化、Login画面への遷移

成功時

項目	内容
HTTP	204 No Content
Response Body	なし
Cookie	Refresh TokenとCSRFを削除
Frontend状態	未認証へ戻す

既に失効済みのTokenに対するLogoutは、再送可能な処理として成功扱いにする。

完了条件

- Logout後に同じRefresh Tokenを使用できない。
- Cookieが削除される。
- AuthContextが初期化される。
- Login画面へ戻る。
- 保護画面を再表示できない。

現在User取得仕様

フロー

1. FrontendがAccess TokenをAuthorization Headerへ設定する。
2. `GET /me` を呼び出す。
3. Auth MiddlewareがJWT署名を検証する。
4. Auth MiddlewareがJWT有効期限を確認する。
5. JWTからUser IDを取得する。
6. `UserUsecase.GetMe`を呼び出す。
7. DBからUserを取得する。
8. `User.Status`が `active` であることを確認する。
9. JWT内とDB内のTokenVersionを比較する。
10. 正常なUserをEcho Contextへ保存する。
11. ControllerがUser情報を返す。
12. AuthContextへUserを保存する。

Auth Middlewareで確認するもの

- Authorization Header
 - Bearer形式
 - JWT署名
 - 有効期限
 - User ID
 - `User.Status`
 - `TokenVersion`
-

Frontend仕様

実装順

1. `types/user.ts`
2. `api/client.ts`
3. `api/user.ts`
4. `auth/AuthContext.tsx`
5. `auth/ProtectedRoute.tsx`
6. `pages/SignupPage.tsx`
7. `pages/LoginPage.tsx`
8. `pages/TemporaryHomePage.tsx`

9. `pages/NotFoundPage.tsx`
 10. `router/AppRouter.tsx`
 11. `App.tsx`
-

types/user.ts

Backend APIとの受け渡しに必要な型を定義する。

- User
- SignUp入力
- Login入力
- Login結果
- API Error
- AuthContextの状態

具体的な型名は、BackendのResponseと統一する。

api/client.ts

次の共通処理をまとめる。

1. API Base URL
2. JSON Header
3. Cookie送信
4. Authorization Header
5. CSRF Header
6. APIエラー変換
7. 401受信時のRefresh
8. Refresh後の元Request再送
9. Refreshの多重実行防止

401処理

1. 通常APIが401を返す。
2. Refresh API自身ではないことを確認する。
3. Refresh APIを1回だけ呼び出す。
4. 成功した場合はAccess Tokenを更新する。
5. 元Requestを1回だけ再送する。
6. 再送も失敗した場合は未認証状態へ戻す。

7. 無限にRefreshを繰り返さない。
8. 複数APIが同時に401になってもRefresh APIを1回だけ実行する。

api/user.ts

次のAPI通信をまとめる。

- SignUp
- Login
- Refresh
- Logout
- Me

AuthContext

保持する状態

項目	内容
<code>user</code>	Login User
<code>accessToken</code>	API認証用Token
<code>isAuthenticated</code>	認証済みか
<code>isLoading</code>	認証確認中か

提供する処理

処理	内容
<code>login</code>	Loginし、認証状態を保存する
<code>logout</code>	Logoutし、認証状態を削除する
<code>refresh</code>	Access Tokenを再発行する
<code>restore</code>	ページ読込時に認証を復元する

初期化フロー

1. アプリ起動時は認証確認中とする。
2. Refresh Token CookieはJavaScriptから読み取らない。
3. CSRF Cookieがある場合はRefresh APIを呼び出す。
4. Refresh成功時にAccess Tokenを保持する。
5. `GET /me` を呼び出す。
6. User情報を保持する。

7. 認証済み状態へ変更する。
 8. Refresh失敗時はUserとAccess Tokenを削除する。
 9. 未認証状態へ変更する。
 10. React Routerが表示画面を決定する。
-

ProtectedRoute

認証状態	処理
確認中	Loading表示
認証済み	対象画面を表示
未認証	Login画面へ遷移

認証確認前に保護画面を表示しない。

SignupPage

URL

`/signup`

表示内容

- Name
- Email
- Password
- 登録ボタン
- Login画面へのリンク
- エラー表示

フロー

1. Userが入力する。
 2. 送信中は登録ボタンを無効化する。
 3. SignUp APIを呼び出す。
 4. 成功時は `/login` へ遷移する。
 5. 失敗時は入力またはAPIエラーを表示する。
 6. PasswordをConsoleへ出力しない。
-

LoginPage

URL

`/login`

表示内容

- Email
- Password
- Loginボタン
- SignUp画面へのリンク
- エラー表示

フロー

1. Userが入力する。
 2. 送信中はLoginボタンを無効化する。
 3. Login APIを呼び出す。
 4. AuthContextへAccess TokenとUserを保存する。
 5. 成功時は `/` へ遷移する。
 6. 失敗時は認証エラーを表示する。
-

TemporaryHomePage

URL

`/`

表示内容

- User名
- Email
- Role
- Logoutボタン

動画関連の表示は含めない。

フロー

1. ProtectedRouteが認証済みであることを確認する。
2. AuthContextのUser情報を表示する。
3. Logoutボタンを押す。
4. Logout APIを呼び出す。

5. AuthContextを初期化する。
6. `/login` へ遷移する。

NotFoundPage

存在しないURLへアクセスした場合にNot Foundを表示する。

React Router

URL	画面	公開範囲
<code>/signup</code>	SignupPage	未認証可
<code>/login</code>	LoginPage	未認証可
<code>/</code>	TemporaryHomePage	認証済み限定
その他	NotFoundPage	全員

フロー

1. アプリを起動する。
2. AuthContextが認証状態を確認する。
3. 確認中はLoadingを表示する。
4. SignupPageとLoginPageは未認証でも表示する。
5. TemporaryHomePageはProtectedRouteで保護する。
6. 認証済みならTemporaryHomePageを表示する。
7. 未認証ならLoginPageへ遷移する。
8. 存在しないURLではNotFoundPageを表示する。

App.tsx

1. AuthContextをFrontend全体へ提供する。
2. AppRouterを配置する。
3. AuthContextの初期化中はLoading表示を行う。

XSS対策

Frontendでは、User名やEmailをHTMLとして実行せず、文字列として表示する。
次を行わない。

- User入力を `dangerouslySetInnerHTML` へ直接渡す。
- User入力をHTML文字列として挿入する。

- User入力に含まれるscriptやイベント属性を実行する。

完了条件は、User入力にHTMLやscript形式の文字列が含まれていても、コードとして実行されないこととする。

OpenAPI整合性

実装後は、次をOpenAPIと一致させる。

1. APIパス
2. HTTPメソッド
3. Request項目
4. Response項目
5. Cookie
6. Authorization Header
7. CSRF Header
8. HTTPステータス
9. APIエラー形式

対象APIは次の5つとする。

- `POST /signup`
- `POST /login`
- `POST /refresh`
- `POST /logout`
- `GET /me`

テスト仕様

Backend

対象	主な確認
User Entity	初期状態、Role、Status、TokenVersion
RefreshToken Entity	期限、使用済み、失効、MarkUsed
UserRepository	Create、FindByEmail、FindByID、Update、Email重複
RefreshTokenRepository	Create、Rotate、Family失効、行ロック、Rollback
RateLimitRepository	Lua Script、補充、超過、同時実行
SignUp Usecase	正常、Email重複、Hash失敗
Login Usecase	正常、Password不一致、suspended

対象	主な確認
Refresh Usecase	Rotation、再利用検知、排他制御
Logout Usecase	Family失効、再送
Controller	HTTP、Cookie、エラー形式
Rate Limit Middleware	許可、制限超過
CSRF Middleware	正常、Cookieなし、Headerなし、不一致
Auth Middleware	JWT、期限、TokenVersion、Status
Validator	境界値、Email形式、必須入力

Frontend

対象	主な確認
SignupPage	入力、送信、エラー、遷移
LoginPage	Login、認証失敗、遷移
AuthContext	Login、Refresh、Logout、復元
ProtectedRoute	確認中、認証済み、未認証
TemporaryHomePage	User表示、Logout
API Client	Authorization、CSRF、Refresh、再送制限
AppRouter	URLと画面の対応

Backend確認

Frontendへ進む前に、次を確認する。

1. PostgreSQLを起動する。
2. Redisを起動する。
3. Migrationを実行する。
4. usersテーブルを確認する。
5. refresh_tokensテーブルを確認する。
6. Backendテストを実行する。
7. Backendを起動する。
8. `POST /signup`を確認する。
9. `POST /login`を確認する。
10. `POST /refresh`を確認する。
11. `GET /me`を確認する。
12. `POST /logout`を確認する。
13. CSRF不一致時の拒否を確認する。

14. Rate Limit超過時の拒否を確認する。

Backend APIが正常に動作してからFrontend実装へ進む。

ブラウザ確認

起動条件

1. PostgreSQLが起動している。
 2. Redisが起動している。
 3. Migrationが完了している。
 4. Backendが起動している。
 5. Frontendが起動している。
 6. Frontend OriginがCORSで許可されている。
 7. Cookie設定が実行環境に合っている。
 8. FrontendのAPI接続先がBackendを向いている。
-

会員登録

1. `/signup` を開く。
 2. Name、Email、Passwordを入力する。
 3. 登録ボタンを押す。
 4. `/login` へ遷移する。
 5. usersテーブルへUserが1件保存される。
 6. Roleが `user` である。
 7. Statusが `active` である。
 8. Passwordが平文保存されていない。
-

Login

1. `/login` を開く。
2. 登録済みEmailとPasswordを入力する。
3. Loginする。
4. `/` へ遷移する。
5. User名、Email、Roleが表示される。
6. Refresh Token CookieがHttpOnlyである。

7. CSRF Cookieが存在する。
-

ページ再読み込み

1. `/` でページを再読み込みする。
 2. Refresh APIが呼ばれる。
 3. 新しいAccess Tokenが返る。
 4. `GET /me` が呼ばれる。
 5. User情報が再表示される。
 6. Login画面へ戻らない。
-

ProtectedRoute

1. 認証Cookieがない状態にする。
 2. `/` へ直接アクセスする。
 3. 認証確認中はLoadingを表示する。
 4. 未認証確定後に `/login` へ遷移する。
 5. TemporaryHomePageが一瞬表示されない。
-

CSRF

1. `X-CSRF-Token` を付けずにRefresh APIを呼ぶ。
 2. 403になる。
 3. 不正なCSRF TokenでLogout APIを呼ぶ。
 4. 403になる。
 5. 正しいCSRF Tokenでは処理できる。
-

Rate Limit

1. SignUp、Login、Refreshを制限回数以上実行する。
 2. 429になる。
 3. `Retry-After` が返る。
 4. 待機後に再び処理できる。
-

Refresh Token Rotation

1. Loginする。

2. Refresh APIを呼ぶ。
 3. Refresh Token Cookieが交換される。
 4. 旧Refresh Tokenを再利用する。
 5. Access Tokenを再発行できない。
 6. 同じFamilyIDのRefresh Tokenを利用できない。
 7. 既存Access TokenがTokenVersion不一致で拒否される。
-

Logout

1. Loginする。
2. Logoutボタンを押す。
3. Refresh Token Cookieが削除される。
4. CSRF Cookieが削除される。
5. AuthContextが未認証状態になる。
6. `/login` へ遷移する。
7. ブラウザの戻る操作でも `/` を表示できない。
8. Logout前のRefresh Tokenで再発行できない。